AEP-X — Microservices Architecture Implementation Guide

Turning the Volume 9 platform blueprint into a running, dependency-ordered microservices system

Source AEP-X Ultra.docx — 12-Volume Reference Series outline (Vol. 3 & 9)

Companion to ADLC-Plan, Handoff, Instructional-Manual **Prepared** 7 July 2026

Contents

1. Reconciliation — How This Source Relates to the Existing Plan

What this document is: a 12-volume, ~2,000–2,500 page professional reference series outline (author: Ali Abdalla). Only Volume I (Foundations) and Volume II (Vision/Strategy/PESTEL) are written to full manuscript detail — chapters with topics, case studies, and exercises. Volumes III–XII, including **Volume 9: Platform Implementation**, exist only as chapter-title outlines at this stage. This is a book, not a delivery plan — it names *what* the platform's services are, not *when* or *in what order* to build them.

This guide does the engineering work the book doesn't attempt: it takes Volume 9's named service catalogue and Volume 3's 15-Layer AEP-X Stack, and turns them into a concrete, dependency-ordered, bounded-context microservices architecture — reconciled against the scope already committed in the Handoff document.

1.1 What's new in this source vs. the original manual

Volume 9 chapter	Names
Ch. 98–102 (Data layer)	Reference Monorepo, PostgreSQL Design, Redis Architecture, Vector Databases, Knowledge Graph Databases
Ch. 103–110 (Services)	Agent Registry, Trust Authority, Workflow Engine, Memory Service, Discovery Service, Safety Engine, Governance Engine, Marketplace Engine

This is a cleaner, 8-service restatement of the original manual's 13-service list — **Trust Authority** merges Identity+Trust, **Governance Engine** merges Policy+Audit — plus it explicitly adds a **Knowledge Graph Database** (Neo4j) to the data layer, which the Instructional Manual only touched via pgvector.

1.2 Cross-walk against the committed scope

Volume 9 service	Status against Handoff §2 / Instructional Manual
Agent Registry	BUILT — Instructional Manual §3.3, unchanged
Trust Authority	BUILT — Identity + Trust stubs, Instructional Manual §3.4, unchanged
Memory Service	BUILT — Instructional Manual §3.5, unchanged
Discovery Service	NEW IN THIS GUIDE — was requirement-level only (Instructional Manual §4.1); full implementation in §5.1 below
Workflow Engine	NEW IN THIS GUIDE — was requirement-level only (§4.2); full implementation with event publishing in §5.2
Safety Engine	NEW IN THIS GUIDE — was a response-contract stub (§4.4); full implementation in §5.3
Governance Engine	NEW IN THIS GUIDE — not previously built; merges the deferred Policy Engine + Audit Service; §5.4
Marketplace Engine	BACKLOG — **explicitly deferred** in Handoff §2. Reference design only, §5.5. Do not build in Weeks 1–12 without re-opening that scope decision.

Scope discipline check: this source names 8 services as a flat list with no priority or timeline — a book outline has no reason to sequence them. If you build straight from Volume 9 without this guide's dependency ordering and without honoring the Handoff's deferral of Marketplace Engine, you will re-create exactly the scope-creep problem the Critical Evaluation (ADLC Plan §15) flagged: everything looks equally urgent, so everything gets started, so nothing finishes. The build sequence in §5 is deliberately ordered so Marketplace Engine comes last and is clearly marked optional.

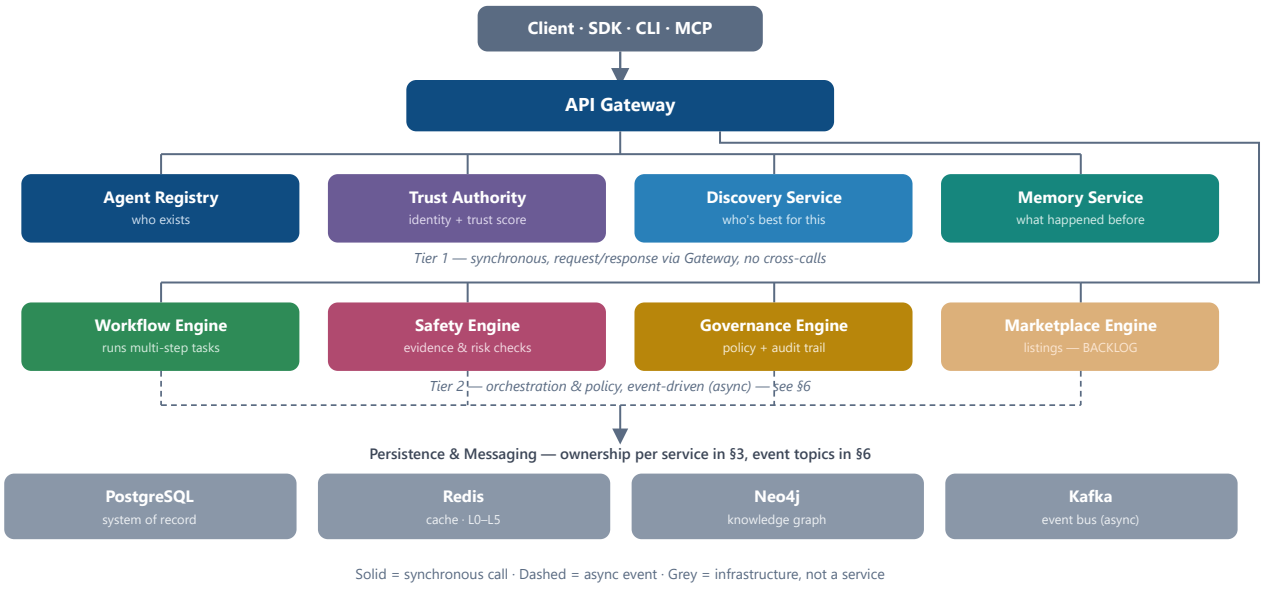
ARCHITECTURE

2. Microservices Architecture Overview

8 services · 1 gateway · 3 data stores · 1 event bus

Three tiers, two communication modes:

- **Tier 1 — Foundational (synchronous):** Agent Registry, Trust Authority, Discovery Service, Memory Service. Stateless-ish, fast, called directly through the Gateway, request/response only. No service in this tier calls another service in this tier.
- **Tier 2 — Orchestration (event-driven):** Workflow Engine, Safety Engine, Governance Engine, Marketplace Engine. Callable through the Gateway too, but their real work happens by publishing and consuming events on the bus — this is where cross-cutting concerns (audit, safety escalation, trust feedback) live without services calling each other directly.
- **Tier 3 — Infrastructure:** PostgreSQL (system of record), Redis (cache), Neo4j (knowledge graph), Kafka (event bus). No service reaches into another service's tables — each service owns its own schema.



Why the Marketplace Engine box is faded: it's on the diagram because Volume 9 names it, but it's backlog per Handoff §2. Building it before the other 7 are live and gated (Trust Authority + Governance Engine both healthy) would let unvetted listings publish with no trust or policy check — the exact failure mode RFC-0002/RFC-0006 exist to prevent.

BOUNDED CONTEXTS

3. Service Catalogue

What each service owns, exposes, and depends on

Service	Owns (bounded context)	Exposes	Depends on	Data store
Agent Registry	Agents, capabilities	`POST/GET /agents`, `/capabilities`	—	PostgreSQL (own schema: `registry.*`)
Trust Authority	Identity, trust scores	`POST /token`, `GET /trust/{id}`	—	PostgreSQL (`trust.*`)
Discovery Service	Nothing persistent — a stateless ranking function	`GET /discover`	Agent Registry, Trust Authority (read-only, via API)	Redis (ranked-result cache only)
Memory Service	Session / episodic / semantic memory entries	`POST/GET /memory/*`, `/memory/search`	—	PostgreSQL + pgvector (`memory.*`), Redis (session TTL)
Workflow Engine	Workflow definitions, run state, step results	`POST /workflows`, `/workflows/{id}/execute`	Agent Registry, Discovery, Memory (via API); Neo4j (knowledge lookups during execution)	PostgreSQL (`workflow.*`); publishes to Kafka
Safety Engine	Risk assessments, evidence validation records	`POST /safety/validate`	Trust Authority (read); consumes `workflow.completed` events	PostgreSQL (`safety.*`); publishes to Kafka
Governance Engine	Policies, immutable audit log	`POST /policy/evaluate`, `GET /audit`	Consumes *every* event topic (by design — audit is cross-cutting)	PostgreSQL, append-only (`governance.*`)
Marketplace Engine *(backlog)*	Listings, publisher accounts	`POST /marketplace/listings`	Agent Registry, Trust Authority, Governance Engine (publish-time checks)	PostgreSQL (`marketplace.*`)

The one rule that makes this a microservices architecture and not a distributed monolith: every row above owns its own tables. If Workflow Engine needs to know an agent's trust score, it calls Trust Authority's API — it never queries `trust.*` tables directly, even though they're in the same PostgreSQL instance in this reference deployment. Enforce this with separate database roles/schemas per service from day one; it's much harder to retrofit later.

PATTERNS

4. Communication & Data-Ownership Patterns

4.1 Synchronous — Tier 1

Gateway → service, plain REST, JSON, <100ms budget end-to-end (ADLC Plan §9). No retries beyond one, no circuit breaker complexity yet — at this scale, a failed Tier-1 call should just surface as a 503 to the caller. Add resilience libraries only once you have production traffic data showing you need them.

4.2 Asynchronous — Tier 2

Workflow Engine, Safety Engine, and Governance Engine communicate mostly by publishing facts, not by calling each other:

- Workflow Engine finishes a run → publishes `workflow.completed`
- Safety Engine consumes it, validates evidence → publishes `safety.flagged` only if something's wrong
- Governance Engine consumes *both* topics unconditionally, for the audit trail — it doesn't decide anything, it just records
- Trust Authority optionally consumes `safety.flagged` to adjust a trust score (the feedback loop from ADLC Plan's ML Integration Layer) — this is the one place an event flows back into a Tier-1 service, and it's a fire-and-forget adjustment, not a synchronous call

This is why Governance Engine can implement 100% audit coverage without every other service having to remember to call it — it's a passive listener, not a dependency anyone codes against.

4.3 Database-per-service, enforced

```
-- Run once per service against the shared Postgres instance in dev.
-- In production, these become separate instances/clusters.
CREATE SCHEMA registry;    CREATE ROLE registry_svc  LOGIN PASSWORD '...';
CREATE SCHEMA trust;       CREATE ROLE trust_svc      LOGIN PASSWORD '...';
CREATE SCHEMA memory;      CREATE ROLE memory_svc     LOGIN PASSWORD '...';
CREATE SCHEMA workflow;    CREATE ROLE workflow_svc   LOGIN PASSWORD '...';
CREATE SCHEMA safety;      CREATE ROLE safety_svc     LOGIN PASSWORD '...';
CREATE SCHEMA governance;  CREATE ROLE governance_svc LOGIN PASSWORD '...';

GRANT ALL ON SCHEMA registry TO registry_svc;
-- repeat per schema/role — never grant a service's role rights on another's schema
```

Each service's `DATABASE_URL` env var connects as its own role, scoped to its own schema. This is enforced at the database level, not just by convention — a bug in Workflow Engine's code physically cannot read Trust Authority's tables.

WEEKS 5–10

5. Step-by-Step Build Sequence

Continues from Instructional Manual Part 4 (Alpha Build) · dependency-ordered

Agent Registry, Trust Authority, and Memory Service are already built (Instructional Manual §3.3–3.5) — no changes needed here. This Part builds the remaining five, in dependency order, and extends the docker-compose stack to add Neo4j and Kafka.

5.0 Extend the infrastructure stack

0

DOCKER-COMPOSE.YML — ADDITIONS

```
neo4j:
  image: neo4j:5-community
  environment:
    - NEO4J_AUTH=neo4j/aepx_dev_only
  ports: ["7474:7474", "7687:7687"]
  healthcheck:
    test: ["CMD-SHELL", "wget -q --spider http://localhost:7474"]
    interval: 10s
    retries: 5

kafka:
  image: confluentinc/cp-kafka:7.6.0
  environment:
    - KAFKA_NODE_ID=1
    - KAFKA_PROCESS_ROLES=broker,controller
    - KAFKA_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
    - KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092
    - KAFKA_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
    - KAFKA_CONTROLLER_LISTENER_NAMES=CONTROLLER
    - CLUSTER_ID=aepx-dev-cluster
  ports: ["9092:9092"]

discovery:
  build: ./services/discovery
  ports: ["8006:8000"]
  depends_on: [registry, trust, redis]

workflow:
  build: ./services/workflow
  ports: ["8007:8000"]
  depends_on: [registry, memory, neo4j, kafka]

safety:
  build: ./services/safety
  ports: ["8008:8000"]
  depends_on: [trust, kafka]

governance:
  build: ./services/governance
  ports: ["8009:8000"]
  depends_on: [kafka]
```

Add `kafka-python==2.0.2` (or `aiokafka==0.11.0` for async) and `neo4j==5.20.0` to the relevant services' `requirements.txt`.

5.1 Discovery Service

1. Stateless ranking function: `40% capability match + 25% trust + 15% availability − 10% latency − 10% cost` (ADLC Plan §6.1 / RFC-0004). Reads from Registry and Trust Authority, caches ranked results.

SERVICES/DISCOVERY/APP/MAIN.PY

```
from fastapi import FastAPI
import httpx, redis, os, json

app = FastAPI(title="AEP-X Discovery Service", version="0.1.0")
r = redis.Redis(host=os.getenv("REDIS_HOST", "redis"), decode_responses=True)
REGISTRY_URL = os.getenv("REGISTRY_URL", "http://registry:8000")
TRUST_URL = os.getenv("TRUST_URL", "http://trust:8000")

@app.get("/health")
def health():
    return {"status": "ok", "service": "discovery"}

def score(agent: dict, trust: dict) -> float:
    capability_match = agent.get("confidence", 0.8)
    trust_score = trust.get("trust_score", 50) / 100
    availability = 1.0  # placeholder until health-aggregation feeds this
    latency_penalty = agent.get("latency_ms", 50) / 1000
    cost_penalty = agent.get("cost", 0.0)
    return (0.40 * capability_match + 0.25 * trust_score + 0.15 * availability
            - 0.10 * latency_penalty - 0.10 * cost_penalty)

@app.get("/discover")
async def discover(capability: str):
    cache_key = f"discover:{capability}"
    cached = r.get(cache_key)
    if cached:
        return {"results": json.loads(cached), "cache_hit": True}

    async with httpx.AsyncClient(timeout=3.0) as client:
        agents = (await client.get(f"{REGISTRY_URL}/agents")).json()
        ranked = []
        for agent in agents:
            trust = (await client.get(f"{TRUST_URL}/trust/{agent['id']}")).json()
            ranked.append({**agent, "score": score(agent, trust)})
        ranked.sort(key=lambda a: a["score"], reverse=True)

    r.set(cache_key, json.dumps(ranked), ex=30)
    return {"results": ranked, "cache_hit": False}
```

> **DoD:** `GET /discover?capability=generate_lesson` returns a ranked list; second call within 30s is a cache hit; p95 latency <50ms uncached, <10ms cached.

5.2 Workflow Engine

2 Sequential orchestration only at this stage (Parallel/Conditional/Dynamic modes are Beta-phase, per Instructional Manual §4.2). Publishes a Kafka event when a run completes — this is the first service in the build that talks to the event bus.

SERVICES/WORKFLOW/APP/MAIN.PY

```
from fastapi import FastAPI
from pydantic import BaseModel
import httpx, json, uuid, time
from kafka import KafkaProducer

app = FastAPI(title="AEP-X Workflow Engine", version="0.1.0")
producer = KafkaProducer(bootstrap_servers="kafka:9092",
                         value_serializer=lambda v: json.dumps(v).encode())

REGISTRY_URL, MEMORY_URL = "http://registry:8000", "http://memory:8000"
_WORKFLOWS: dict[str, dict] = {}

class WorkflowIn(BaseModel):
    name: str
    steps: list[dict]   # [{"capability": "...", "agent_id": "..."}]

@app.get("/health")
def health():
    return {"status": "ok", "service": "workflow"}

@app.post("/workflows")
def create_workflow(w: WorkflowIn):
    wf_id = str(uuid.uuid4())
    _WORKFLOWS[wf_id] = {"id": wf_id, "status": "CREATED", **w.model_dump()}
    return _WORKFLOWS[wf_id]

@app.post("/workflows/{wf_id}/execute")
async def execute_workflow(wf_id: str):
    wf = _WORKFLOWS[wf_id]
    wf["status"] = "RUNNING"
    results = []
    async with httpx.AsyncClient(timeout=5.0) as client:
        payload = {}
        for step in wf["steps"]:
            agent = await client.get(f"{REGISTRY_URL}/agents/{step['agent_id']}")
            result = {"capability": step["capability"], "agent": agent.json(), "input":
payload}
            results.append(result)
            payload = result  # thread output of step N into step N+1

    wf["status"], wf["results"] = "COMPLETED", results
    producer.send("workflow.completed", {
        "workflow_id": wf_id, "status": "COMPLETED",
        "step_count": len(results), "ts": time.time(),
    })
    return wf
```

DoD: a 2-step workflow executes end-to-end; a `workflow.completed` message lands on the Kafka topic (verify with `kafka-console-consumer`); workflow start <100ms, planning <200ms.

5.3 Safety Engine

3 Two entry points: a synchronous `/safety/validate` for direct calls, and a background Kafka consumer that reacts to every completed workflow. Implements the response contract from ADLC Plan §2: `{answer, confidence, evidence[], verification_status}`.

SERVICES/SAFETY/APP/MAIN.PY

```
from fastapi import FastAPI
from pydantic import BaseModel
from kafka import KafkaConsumer, KafkaProducer
import json, threading

app = FastAPI(title="AEP-X Safety Engine", version="0.1.0")
producer = KafkaProducer(bootstrap_servers="kafka:9092",
                         value_serializer=lambda v: json.dumps(v).encode())

class ValidateIn(BaseModel):
    answer: str
    evidence: list[str] = []

CONFIDENCE_BANDS = [(95, "Verified"), (80, "High"), (60, "Medium"), (0, "Unverified")]

def band_for(evidence_count: int) -> str:
    score = min(100, evidence_count * 35)
    for threshold, label in CONFIDENCE_BANDS:
        if score >= threshold:
            return label
    return "Unverified"

@app.get("/health")
def health():
    return {"status": "ok", "service": "safety"}

@app.post("/safety/validate")
def validate(v: ValidateIn):
    verification_status = band_for(len(v.evidence))
    if verification_status == "Unverified":
        producer.send("safety.flagged", {"reason": "no_evidence", "answer": v.answer[:200]})
    return {"answer": v.answer, "confidence": len(v.evidence) * 0.35,
            "evidence": v.evidence, "verification_status": verification_status}

def consume_workflow_events():
    consumer = KafkaConsumer("workflow.completed", bootstrap_servers="kafka:9092",
                             value_deserializer=lambda v: json.loads(v.decode()))
    for msg in consumer:
        event = msg.value
        # Alpha-stage rule: any workflow with zero result steps is flagged for review
        if event.get("step_count", 0) == 0:
            producer.send("safety.flagged", {"reason": "empty_workflow", **event})

threading.Thread(target=consume_workflow_events, daemon=True).start()
```

At Alpha, "evidence" is a list of source strings — the full verification pipeline (cross-source correlation, source trust weighting from RFC-0007) is a Beta-phase build. Don't gold-plate this before the Alpha Gate Review has real data.

DoD: `/safety/validate` with 0 evidence items returns `"Unverified"` and produces a `safety.flagged` event; consumer thread stays alive across 10+ test workflow completions.

5.4 Governance Engine

4 Two jobs: evaluate policy on demand, and consume *every* topic unconditionally to build the audit trail. This is the one service allowed to know about all events — that's what makes 100% audit coverage achievable without instrumenting every other service.

SERVICES/GOVERNANCE/APP/MAIN.PY

```
from fastapi import FastAPI
from kafka import KafkaConsumer
import json, threading, time

app = FastAPI(title="AEP-X Governance Engine", version="0.1.0")
_AUDIT_LOG: list[dict] = []          # swap for an append-only Postgres table before Beta
_POLICIES = {"max_risk_level": "S2"} # seed policy — extend via RFC-0006 categories

@app.get("/health")
def health():
    return {"status": "ok", "service": "governance"}

@app.post("/policy/evaluate")
def evaluate_policy(risk_level: str):
    allowed = risk_level <= _POLICIES["max_risk_level"]
    return {"risk_level": risk_level, "allowed": allowed, "policy": _POLICIES}

@app.get("/audit")
def get_audit(limit: int = 50):
    return _AUDIT_LOG[-limit:]

def consume_all_events():
    consumer = KafkaConsumer(
        "workflow.completed", "safety.flagged",
        bootstrap_servers="kafka:9092",
        value_deserializer=lambda v: json.loads(v.decode()),
    )
    for msg in consumer:
        _AUDIT_LOG.append({"topic": msg.topic, "event": msg.value, "recorded_at":
time.time()})

threading.Thread(target=consume_all_events, daemon=True).start()
```

DoD: every `workflow.completed` and `safety.flagged` event appears in `GET /audit` within 1 second of being published; audit write <10ms once backed by Postgres.

5.5 Marketplace Engine — reference design only (backlog)

5

> Do not build this in Weeks 1–12. It is here so the schema exists when the team deliberately re-opens the scope decision (Handoff §2), not as an instruction to start it now.

```
-- schemas/sql/010_marketplace_backlog.sql (reference only — not applied by default)
CREATE TABLE marketplace_listings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    agent_id UUID NOT NULL,           -- FK by reference, not by DB constraint (cross-service)
    publisher_trust_score INT NOT NULL,
    status TEXT DEFAULT 'draft' CHECK (status IN ('draft','review','published','revoked')),
    created_at TIMESTAMPTZ DEFAULT now()
);
-- Publish-time check (application logic, not SQL):
--   1. call Trust Authority -> publisher_trust_score >= 60
--   2. call Governance Engine POST /policy/evaluate -> allowed == true
--   3. only then set status = 'published'
```

CONTRACTS

6. Event Contracts

Topic	Producer	Consumers	Payload shape
`workflow.completed`	Workflow Engine	Safety Engine, Governance Engine	`{workflow_id, status, step_count, ts}`
`safety.flagged`	Safety Engine	Governance Engine, Trust Authority (optional feedback)	`{reason, answer|workflow_id, ...context}`
`agent.registered` *(add when Registry needs it)*	Agent Registry	Discovery Service (cache invalidation)	`{agent_id, name, ts}`
`trust.updated` *(add with the feedback loop)*	Trust Authority	Discovery Service, Governance Engine	`{entity_id, old_score, new_score, reason}`

Only the first two topics are wired in §5's code — the other two are named here so you don't have to invent topic names later when Discovery's cache-invalidation and the trust feedback loop get built. Add them incrementally; don't pre-build consumers for events nothing produces yet.

PATH TO PRODUCTION

7. Path to Kubernetes

Volume 10 territory — do this after the Alpha Gate, not before

7.1 One Helm chart, one values-per-service pattern

1

```
charts/aepx-service/templates/deployment.yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.name }}
spec:
  replicas: {{ .Values.replicas | default 1 }}
  selector: { matchLabels: { app: {{ .Values.name }} } }
  template:
    metadata: { labels: { app: {{ .Values.name }} } }
    spec:
      containers:
        - name: {{ .Values.name }}
          image: "{{ .Values.image }}:{{ .Values.tag }}"
          ports: [{ containerPort: 8000 }]
          env: {{ toYaml .Values.env | nindent 12 }}
          livenessProbe: { httpGet: { path: /health, port: 8000 }, initialDelaySeconds: 5 }
          resources: {{ toYaml .Values.resources | nindent 12 }}
```

One chart, reused for all 8 services — each gets its own `values-<service>.yaml` setting `name`, `image`, and `env`. This avoids maintaining 8 near-identical charts.

7.2 Rollout order matches the build order

2

```
helm install registry ./charts/aepx-service -f values-registry.yaml
helm install trust    ./charts/aepx-service -f values-trust.yaml
helm install memory   ./charts/aepx-service -f values-memory.yaml
helm install discovery ./charts/aepx-service -f values-discovery.yaml
helm install workflow  ./charts/aepx-service -f values-workflow.yaml
helm install safety    ./charts/aepx-service -f values-safety.yaml
helm install governance ./charts/aepx-service -f values-governance.yaml
# marketplace: not installed — backlog
```

Promote via ArgoCD once this works manually — don't wire GitOps before you've confirmed the chart is correct by hand. Add Prometheus scrape annotations and OpenTelemetry env vars (`OTEL_EXPORTER_OTLP_ENDPOINT`) to the shared chart once the observability stack (Instructional Manual §3, ADLC Plan §6) is live.

MILESTONE

8. Definition of Done — Microservices Architecture Milestone

- All 7 active services (Registry, Trust Authority, Discovery, Memory, Workflow, Safety, Governance) report `"status": "ok"` on `/health` through the Gateway.
- Each service connects to Postgres as its own database role, scoped to its own schema — verified by attempting (and failing) a cross-schema query from a service's own role.
- A workflow execution produces a `workflow.completed` event that Governance Engine's audit log captures within 1 second, and that Safety Engine evaluates.
- An empty/zero-evidence workflow produces a `safety.flagged` event, and it appears in the audit log.
- `helm lint` passes for all 7 active services' values files; Marketplace Engine's chart values are *not* created.
- This milestone is recorded as complete in `governance/decisions/`, with an explicit note on whether Marketplace Engine's scope deferral is still in force.